JIISM - Journal Information System Management

Complete Development Guide for Junior Developers

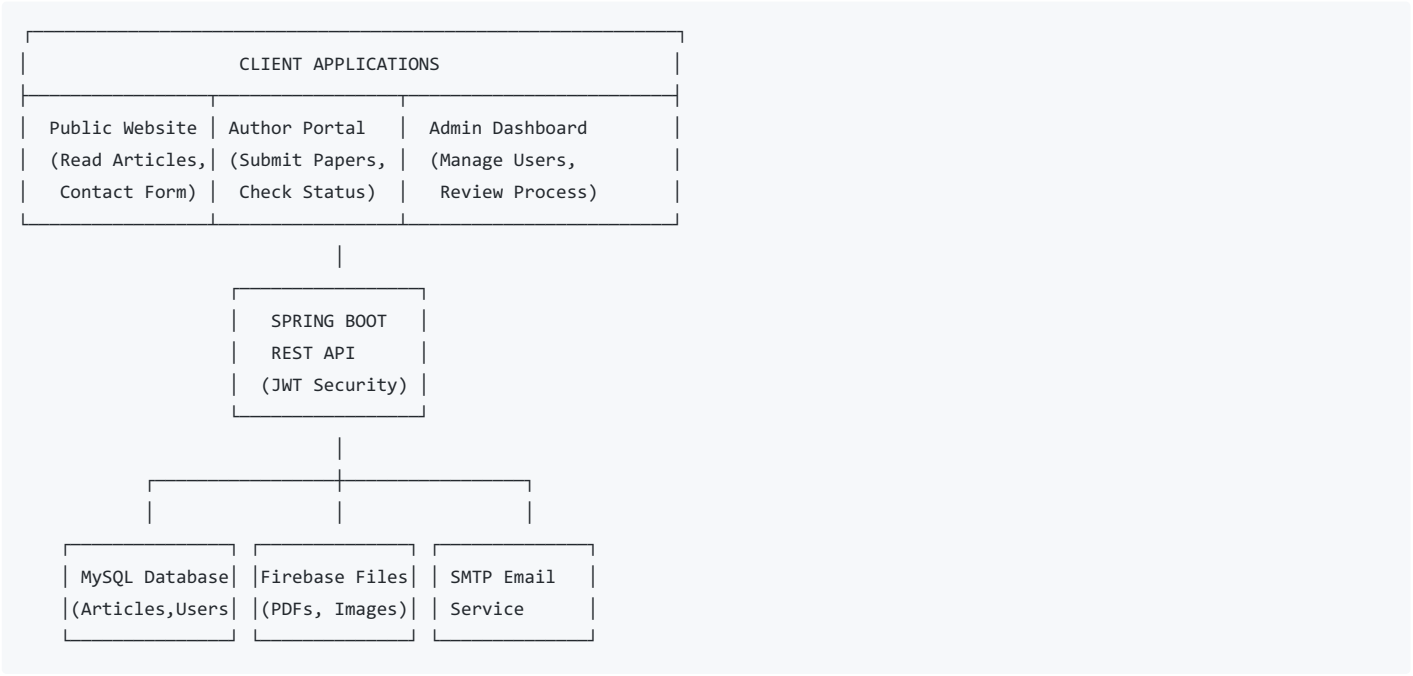
🎯 PROJECT OVERVIEW

What is JIISM? JIISM is a complete academic journal management platform that handles the entire lifecycle of academic articles - from author submission to publication. Think of it as a digital workflow system where authors submit papers, editors review them, and administrators manage the entire process.

Technology Stack:

- **Backend:** Spring Boot 2.4.1 (Java 11)
- **Database:** MySQL 8.0.22
- **Security:** JWT (JSON Web Tokens)
- **File Storage:** Firebase Cloud Storage
- **Frontend Integration:** Angular with NgxAdmin template
- **Email:** SMTP with HTML templates
- **Documentation:** Swagger/OpenAPI

🏗️ SYSTEM ARCHITECTURE & CONCEPTION



Key Design Principles:

1. **Role-Based Access Control:** Different users see different features
2. **RESTful API Design:** Clean, predictable endpoints
3. **Stateless Authentication:** JWT tokens for security
4. **File Security:** Controlled access to uploaded documents
5. **Email Notifications:** Automated workflow communications

📁 PROJECT STRUCTURE EXPLAINED

```
src/main/java/com/accent/jiism/
├── 📁 config/                    # Application Configuration
│   ├── 📄 JwtAuthenticationEntryPoint.java  # Handles auth failures
│   ├── 📄 JwtRequestFilter.java            # Validates JWT on each request
│   ├── 📄 MapperConfiguration.java          # Configures entity-DTO mapping
│   ├── 📄 SwaggerConfig.java               # API documentation setup
│   └── 📄 WebSecurityConfig.java           # Security rules & CORS
│
├── 📁 controller/                # REST API Endpoints (What clients call)
│   ├── 📄 AnnouncementController.java      # News/announcements CRUD
│   ├── 📄 ArticleController.java           # Article management
│   ├── 📄 AuthorityController.java         # User roles/permissions
│   ├── 📄 ContactMessageController.java    # Contact form handling
│   ├── 📄 EditorialMemberController.java   # Editorial board
│   ├── 📄 ExpertiseController.java         # Skills/expertise areas
│   ├── 📄 FileServeController.java         # File download endpoints
│   ├── 📄 FileUploadController.java        # File upload handling
│   ├── 📄 HeroController.java              # Homepage hero section
│   ├── 📄 JournalSectionController.java    # Journal categories
│   ├── 📄 KeyAreaController.java           # Research key areas
│   ├── 📄 SubmissionRequirementController.java # Submission guidelines
│   └── 📄 UserController.java              # User management
│
├── 📁 model/                     # Data Structures
│   ├── 📁 dto/                   # Data Transfer Objects (API communication)
│   │   ├── 📄 ArticleDto.java    # Article data for API
│   │   ├── 📄 UserDto.java       # User data for API
│   │   └── ... (other DTOs)
│   └── 📁 entities/              # Database Tables (JPA Entities)
│       ├── 📄 Article.java       # Article database table
│       ├── 📄 User.java          # User database table
│       └── ... (other entities)
│
├── 📁 repository/                # Database Access Layer
│   ├── 📄 ArticleRepository.java  # Article database operations
│   ├── 📄 UserRepository.java     # User database operations
│   └── ... (other repositories)
│
├── 📁 security/                  # Authentication & Authorization
│   ├── 📁 config/
│   │   └── 📄 JwtTokenUtil.java   # JWT token creation/validation
│   ├── 📁 controller/
│   │   ├── 📄 JwtAuthenificationAdminController.java # Admin login
│   │   └── 📄 JwtAuthenificationController.java      # User login
│   ├── 📁 exception/             # Custom error handling
│   ├── 📁 model/                 # Security-related data models
│   └── 📁 service/
│       └── 📄 JwtUserDetailsService.java # User authentication logic
│
├── 📁 service/                   # Business Logic Layer
│   ├── 📁 serviceImpl/           # Implementation classes
│   └── ... (service interfaces)   # What each service should do
│
├── 📁 types/                     # Enumerations (Predefined Values)
│   ├── 📄 ArticleStatus.java     # PENDING, APPROVED, REJECTED, etc.
│   └── 📄 MessageStatus.java     # READ, UNREAD, ARCHIVED
│
└── 📁 utils/                     # Helper Classes
    ├── 📁 mappers/               # Convert between entities and DTOs
    ├── 📄 ApiResponse.java       # Standardized API response format
    ├── 📄 CurrentUserAuthenticated.java # Get current logged-in user
    ├── 📄 EmailService.java      # Send emails
    └── 📄 ExcelUtils.java        # Excel file processing
```

🗄️ DATABASE DESIGN & ENTITIES

Core Entities

User Entity - Complete Structure

```
@Entity
@Table(name = "user")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private String username;        // Unique login name
    private String email;          // Email address
    private String password;       // Encrypted password
    private String firstName;      // User's first name
    private String lastName;       // User's last name
    private String phone;          // Phone number

    // Enhanced fields for better user management
    private Boolean emailVerified = false;    // Email confirmation status
    private String verificationToken;         // Email verification token
    private String resetPasswordToken;        // Password reset token
    private Timestamp resetPasswordExpires;   // Token expiration
    private Timestamp lastLogin;              // Last login tracking
    private Boolean isActive = true;          // Account active status
    private Timestamp createdAt;              // Registration date
    private Timestamp updatedAt;              // Last profile update

    // Relationships
    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(name = "user_authority",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "authority_id"))
    private Set<Authority> authorities = new HashSet<>();

    @OneToOne(mappedBy = "user", cascade = CascadeType.ALL)
    private UserProfile profile;
}
```

Article Entity - Enhanced Structure

```
@Entity
@Table(name = "articles")
public class Article {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;           // Article title
    private String abstractText;    // Article abstract
    private String keywords;        // Comma-separated keywords
    private String authors;         // Author names
    private String affiliations;    // Author institutions

    // File management
    private String pdfUrl;          // PDF file location
    private String thumbnailUrl;    // Article thumbnail

    // Metrics
    private Integer downloadCount = 0;  // Download tracking
    private Integer consultCount = 0;   // View tracking

    // Workflow
    @Enumerated(EnumType.STRING)
    private ArticleStatus status = ArticleStatus.PENDING;

    // Timestamps
    private Timestamp submittedAt;  // Submission date
    private Timestamp publishedAt;  // Publication date
    private Timestamp lastModified; // Last update

    // Relationships
    @ManyToOne
    @JoinColumn(name = "submitted_by")
    private User submittedBy;       // Author who submitted

    @ManyToOne
    @JoinColumn(name = "journal_section_id")
    private JournalSection section; // Journal section

    @OneToMany(mappedBy = "article", cascade = CascadeType.ALL)
    private List<ArticleReview> reviews = new ArrayList<>();

    @OneToMany(mappedBy = "article", cascade = CascadeType.ALL)
    private List<ArticleAssignment> assignments = new ArrayList<>();
}
```

New Required Entities

ArticleReview Entity - Review System

```
@Entity
@Table(name = "article_reviews")
public class ArticleReview {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name = "article_id", nullable = false)
    private Article article;        // Article being reviewed

    @ManyToOne
    @JoinColumn(name = "editor_id", nullable = false)
    private User editor;            // Reviewing editor

    @Enumerated(EnumType.STRING)
    private ReviewDecision decision; // ACCEPT, MINOR_REVISION, MAJOR_REVISION, REJECT

    @Column(columnDefinition = "TEXT")
    private String reviewComments;   // Detailed feedback

    private Timestamp reviewedAt;    // Review completion date
    private Boolean isVisible = true; // Soft delete flag
}

enum ReviewDecision {
    ACCEPT,
    MINOR_REVISION,
    MAJOR_REVISION,
    REJECT
}
```

ArticleAssignment Entity - Editor Assignment System

```
@Entity
@Table(name = "article_assignments")
public class ArticleAssignment {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name = "article_id", nullable = false)
    private Article article;        // Assigned article

    @ManyToOne
    @JoinColumn(name = "editor_id", nullable = false)
    private User editor;            // Assigned editor

    @ManyToOne
    @JoinColumn(name = "assigned_by_user_id", nullable = false)
    private User assignedBy;        // Admin who made assignment

    private Timestamp assignedAt;   // Assignment date
    private Timestamp dueDate;      // Review deadline

    @Enumerated(EnumType.STRING)
    private AssignmentStatus status = AssignmentStatus.ASSIGNED;
}

enum AssignmentStatus {
    ASSIGNED,    // Editor has been notified
    IN_REVIEW,   // Editor is actively reviewing
    COMPLETED    // Review has been submitted
}
```

EmailTemplate Entity - Email Management

```
@Entity
@Table(name = "email_templates")
public class EmailTemplate {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private String templateName;    // Unique template identifier
    private String subject;         // Email subject line

    @Column(columnDefinition = "TEXT")
    private String body;           // HTML email body

    @Column(columnDefinition = "JSON")
    private String variables;      // Available template variables

    private Boolean active = true; // Template active status
    private Timestamp createdAt;
    private Timestamp updatedAt;
}
```

AUTHENTICATION & SECURITY SYSTEM

How JWT Works in JIISM

```
// 1. User logs in with username/password
POST /authenticate
{
    "username": "john.doe",
    "password": "securepassword"
}

// 2. Server validates and returns JWT token
{
    "success": true,
    "token": "eyJhbGciOiJIUzI1NiIs...",
    "user": { "id": 1, "username": "john.doe", "roles": ["AUTHOR"] }
}

// 3. Client includes token in all subsequent requests
Headers: {
    "Authorization": "Bearer eyJhbGciOiJIUzI1NiIs..."
}

// 4. Server validates token and grants access
```

User Roles & Permissions

```
public enum UserRole {
    ADMIN("ROLE_ADMIN"),     // Full system access
    EDITOR("ROLE_EDITOR"),   // Review articles, manage assignments
    AUTHOR("ROLE_AUTHOR");   // Submit articles, view own submissions
}
```

Permission Matrix:

Action	ADMIN	EDITOR	AUTHOR	Public
View Published Articles	✅	✅	✅	✅
Submit Articles	✅	✅	✅	❌
Review Articles	✅	✅	❌	❌
Manage Users	✅	❌	❌	❌
System Settings	✅	❌	❌	❌

🔌 API ENDPOINTS DOCUMENTATION

🌐 Public APIs (No Authentication Required)

Content Browsing APIs

```
// Get all published articles
GET /api/articles/status/PUBLISHED
Response: {
    "success": true,
    "data": [
        {
            "id": 1,
            "title": "Machine Learning in Healthcare",
            "authors": "Dr. John Smith, Dr. Jane Doe",
            "abstractText": "This paper explores...",
            "downloadCount": 145,
            "consultCount": 1203,
            "publishedAt": "2024-01-15T10:30:00Z"
        }
    ]
}

// Get recent announcements
GET /api/announcements/recent
// Get editorial board members
GET /api/editorial-members
// Get research key areas
GET /api/key-areas
// Get journal sections
GET /api/journal-sections/active
```

Interaction APIs

```
// Track article views
POST /api/articles/{id}/consult
// Track article downloads
POST /api/articles/{id}/download
// Submit contact form
POST /api/contact-messages
{
    "name": "John Doe",
    "email": "john@example.com",
    "subject": "Question about submission",
    "message": "I would like to know..."
}
```

File Access APIs

```
// Download article PDF
GET /api/files/articles/{filename}.pdf
// View article images
GET /api/files/images/{filename}.jpg
// Access public documents
GET /api/files/public/{filename}
```

🔒 Protected APIs (Authentication Required)

Authentication APIs

```
// User login
POST /authenticate
{
    "username": "author123",
    "password": "mypassword"
}

// Admin login
POST /authenticateadmin
{
    "username": "admin",
    "password": "adminpass"
}

// Author registration
POST /api/users/register/author
{
    "username": "newauthor",
    "email": "author@university.edu",
    "firstName": "Jane",
    "lastName": "Smith",
    "password": "securepass",
    "expertise": ["Machine Learning", "Data Science"]
}
```

IMPLEMENTATION GUIDE FOR MISSING FEATURES

1. AuthController Implementation

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    @Autowired
    private AuthenticationManager authenticationManager;

    @Autowired
    private JwtTokenUtil jwtTokenUtil;

    @Autowired
    private UserService userService;

    @PostMapping("/signin")
    public ResponseEntity<ApiResponse<JwtResponse>> login(@RequestBody JwtRequest request) {
        // 1. Validate username/password
        // 2. Generate JWT token
        // 3. Update last login timestamp
        // 4. Return token and user info
    }

    @PostMapping("/refresh")
    public ResponseEntity<ApiResponse<JwtResponse>> refreshToken(@RequestBody RefreshTokenRequest request) {
        // 1. Validate refresh token
        // 2. Generate new access token
        // 3. Return new token
    }

    @PostMapping("/logout")
    public ResponseEntity<ApiResponse<String>> logout(HttpServletRequest request) {
        // 1. Extract token from request
        // 2. Add token to blacklist (optional)
        // 3. Return success message
    }
}
```

2. Article Workflow Implementation

```
@Service
public class ArticleWorkflowService {

    public ApiResponse<ArticleDto> submitForReview(Long articleId, Long editorId) {
        // 1. Validate article exists and is in correct state
        Article article = articleRepository.findById(articleId);

        // 2. Create assignment record
        ArticleAssignment assignment = new ArticleAssignment();
        assignment.setArticle(article);
        assignment.setEditor(editorRepository.findById(editorId));
        assignment.setAssignedAt(new Timestamp(System.currentTimeMillis()));

        // 3. Update article status
        article.setStatus(ArticleStatus.UNDER_REVIEW);

        // 4. Send email notification to editor
        emailService.sendReviewAssignmentNotification(assignment);

        // 5. Return updated article
        return new ApiResponse<>(true, "Article assigned for review", articleMapper.toDto(article));
    }
}
```

📧 EMAIL SYSTEM IMPLEMENTATION

Email Service Structure

```
@Service
public class EmailServiceImpl implements EmailService {

    @Value("${spring.mail.username}")
    private String fromEmail;

    @Autowired
    private JavaMailSender mailSender;

    @Async
    public void sendWelcomeEmail(User user) {
        EmailTemplate template = emailTemplateRepository.findByName("WELCOME_AUTHOR");
        String htmlContent = processTemplate(template.getBody(), Map.of(
            "firstName", user.getFirstName(),
            "username", user.getUsername(),
            "loginUrl", "https://www.j2ism.org/login"
        ));

        sendHtmlEmail(user.getEmail(), template.getSubject(), htmlContent);
    }

    @Async
    public void sendArticleStatusUpdate(Article article, ArticleStatus oldStatus, ArticleStatus newStatus) {
        EmailTemplate template = emailTemplateRepository.findByName("ARTICLE_STATUS_UPDATE");
        // Process template and send email
    }
}
```

Required Email Templates

1. **WELCOME_AUTHOR** - New author registration
2. **EDITOR_INVITATION** - Editor account setup
3. **ARTICLE_SUBMITTED** - Submission confirmation
4. **REVIEW_ASSIGNED** - Editor notification
5. **ARTICLE_STATUS_UPDATE** - Status change notification
6. **PASSWORD_RESET** - Password recovery

FILE MANAGEMENT SYSTEM

File Upload Implementation

```
@PostMapping("/upload/pdf")
public ResponseEntity<ApiResponse<String>> uploadPdf(@RequestParam("file") MultipartFile file) {
    // 1. Validate file type and size
    if (!file.getContentType().equals("application/pdf")) {
        return ResponseEntity.badRequest()
            .body(new ApiResponse<>(false, "Only PDF files are allowed"));
    }

    if (file.getSize() > 10 * 1024 * 1024) { // 10MB limit
        return ResponseEntity.badRequest()
            .body(new ApiResponse<>(false, "File size must be less than 10MB"));
    }

    // 2. Generate unique filename
    String filename = UUID.randomUUID() + "_" + file.getOriginalFilename();

    // 3. Upload to Firebase
    String downloadUrl = firebaseService.uploadFile(file, "articles/" + filename);

    // 4. Return download URL
    return ResponseEntity.ok(new ApiResponse<>(true, "File uploaded successfully", downloadUrl));
}
```

🧪 TESTING STRATEGY

Unit Testing Example

```
@ExtendWith(MockitoExtension.class)
class ArticleServiceTest {

    @Mock
    private ArticleRepository articleRepository;

    @Mock
    private UserRepository userRepository;

    @InjectMocks
    private ArticleServiceImpl articleService;

    @Test
    void shouldCreateArticleSuccessfully() {
        // Given
        ArticleDto articleDto = new ArticleDto();
        articleDto.setTitle("Test Article");

        User author = new User();
        author.setId(1);

        when(userRepository.findById(1)).thenReturn(Optional.of(author));
        when(articleRepository.save(any(Article.class))).thenAnswer(i -> i.getArguments()[0]);

        // When
        ApiResponse<ArticleDto> result = articleService.createArticle(articleDto);

        // Then
        assertTrue(result.isSuccess());
        assertEquals("Test Article", result.getData().getTitle());
    }
}
```

🚀 DEPLOYMENT & CONFIGURATION

Application Properties

```
 # Database Configuration
spring.datasource.url=jdbc:mysql://localhost:3306/jiism_db?useSSL=false&serverTimezone=UTC
spring.datasource.username=${DB_USERNAME:jiism_user}
spring.datasource.password=${DB_PASSWORD:secure_password}

# JWT Configuration
jwt.secret=${JWT_SECRET:mySecretKey}
jwt.expiration=86400  # 24 hours

# Email Configuration
spring.mail.host=${SMTP_HOST:smtp.gmail.com}
spring.mail.port=${SMTP_PORT:587}
spring.mail.username=${EMAIL_USERNAME:your-email@gmail.com}
spring.mail.password=${EMAIL_PASSWORD:your-app-password}

# File Upload Configuration
spring.servlet.multipart.max-file-size=10MB
spring.servlet.multipart.max-request-size=10MB

# Firebase Configuration
firebase.credentials.path=${FIREBASE_CREDENTIALS:/path/to/firebase-key.json}
firebase.bucket.name=${FIREBASE_BUCKET:jiism-files}
```

✅ DEVELOPMENT CHECKLIST

Phase 1: Authentication Enhancement

- [ ] Implement unified AuthController
- [ ] Add password reset functionality
- [ ] Implement email verification
- [ ] Add refresh token mechanism

Phase 2: Article Workflow

- [ ] Create ArticleAssignment entity and repository
- [ ] Implement editor assignment system
- [ ] Build review submission workflow
- [ ] Add status change notifications

Phase 3: Admin Dashboard

- [ ] Create AdminArticleController
- [ ] Implement user management features
- [ ] Build statistics dashboard
- [ ] Add batch operations

Phase 4: Email System

- [ ] Create EmailTemplate entity
- [ ] Build template processing system
- [ ] Implement all required email notifications
- [ ] Add email tracking and analytics

Phase 5: Testing & Documentation

- [ ] Write unit tests for all services
- [ ] Create integration tests for APIs
- [ ] Update Swagger documentation
- [ ] Create deployment documentation

This comprehensive guide provides everything a junior developer needs to understand and continue development of the JIISM project. Each section builds upon the previous one, ensuring a complete understanding of the system architecture and implementation requirements.